PROJECT BASED EXPERIMENTAL LEARNING

COURSE – NOSQL

PROJECT REPORT

TOPIC – RESTAURANT MENU MANAGEMENT SYSTEM USING MONGODB

SUBMITTED TO
Mr. HARENDRA SINGH

SUBMITTED BY
ANSHU YADAV
BCA (DS & AI)
ROLL NO. · 1250258104
SCHOOL OF COMPUTER APPLICATIONS
BABU BANARASI DAS UNIVERSITY, LUCKNOW

ACADEMIC SESSION 2026–2027

INDEX

Sr. No.	Section
1	Acknowledgement
2	Abstract
3	Introduction and Objectives
4	Methodology
5	Implementation Details
6	MongoDB Operations Demonstrated
7	Results
8	Conclusion and Future Scope
9	References

REPORT BASIS

This report presents the practical work carried out for the NoSQL course through a Restaurant Menu Management System built on MongoDB, a document-oriented NoSQL database.

ACKNOWLEDGEMENT

I sincerely express my gratitude to my teacher Mr. Harendra Singh for giving me the opportunity to work on this NoSQL project titled “Restaurant Menu Management System Using MongoDB”.

This project helped me understand the basic concepts of NoSQL databases and learn how MongoDB is used to create, retrieve, update, sort and delete data. I also learned how query operators such as greater than, less than, equal to, AND and OR are used to filter menu items by price, rating and category.

I am thankful to my teacher for the guidance and support provided throughout the project, and I acknowledge the MongoDB documentation and tools that helped me practise queries and understand document-based data management.

— ASHISH PATEL

ABSTRACT

This report presents a Restaurant Menu Management System developed using MongoDB, a NoSQL document-oriented database. The project shows how the menu of a restaurant can be stored as documents and managed through MongoDB queries.

The system uses a RestaurantDB database and a menu collection containing 50 dish records across five categories: Starters, Main Course, Breads and Rice, Desserts and Beverages. Each record has an item ID, dish name, category, price and customer rating. The project demonstrates database and collection creation, insertion, retrieval, filtering, updating, deletion, sorting, limiting, category-based searching and document counting.

Comparison operators ($eq, $gt, $lt, $gte, $lte) and logical operators ($and, $or) are also demonstrated, giving practical exposure to core NoSQL operations.

KEY FEATURES

- Document-based menu data storage using MongoDB.
- Creation of a database and collection.
- Insertion of 50 menu documents.
- CRUD operations: create, read, update and delete.
- Comparison and logical query operators on price, rating and category.
- Sorting, limiting, category-based searching and counting.

INTRODUCTION

MongoDB is a NoSQL database management system that stores information as flexible documents instead of fixed rows and columns. This makes it well suited to applications such as restaurant menus, where items change often and need to be searched by several attributes.

In this project a menu system for a restaurant has been created using MongoDB. The database holds 50 dishes with their ID, name, category, price and rating. A series of operations is performed on these records to demonstrate practical database management, including filtering by price and rating, combining conditions, revising prices, removing low-rated or discontinued dishes, ordering by price and counting what remains.

OBJECTIVES

- To understand the concept of NoSQL databases.
- To learn the basic features and document model of MongoDB.
- To create a database and collection in MongoDB.
- To insert and manage 50 menu records.
- To perform CRUD operations on menu documents.
- To use comparison operators $eq, $gt, $lt, $gte and $lte.
- To perform logical AND and OR operations.
- To sort, limit, search and count documents.

PROJECT DATA MODEL

Field	Example	Purpose
`itemid`	301	Unique menu item ID
`dish`	Paneer Tikka	Name of the dish
`category`	Starters	Menu category
`price`	280	Price in rupees
`rating`	4.5	Average customer rating out of 5

METHODOLOGY

The project follows a practical sequence of MongoDB operations. The database is first selected, followed by collection creation and insertion of the menu dataset. The stored records are then queried and manipulated using MongoDB operators.

Step	Activity	Description
1	Create Database	Create or switch to the RestaurantDB database using use RestaurantDB.
2	Create Collection	Create a menu collection inside the database.
3	Insert Records	Insert 50 dish documents with ID, name, category, price and rating.
4	Retrieve Data	Display the stored documents using find().
5	Filter Data	Use comparison operators to retrieve records based on price and rating.
6	Update Data	Use updateOne() and updateMany() to revise prices.
7	Delete Data	Use deleteOne() and deleteMany() to remove records.
8	Logical Queries	Use $and and $or to combine conditions.
9	Organize Results	Use sort() and limit() to control query results.
10	Search and Count	Search by category and count documents in the collection.

BASIC DATABASE SETUP

```
use RestaurantDB
db.createCollection("menu")
```

The first command creates or switches to the RestaurantDB database and the second creates the menu collection.

IMPLEMENTATION DETAILS

The implementation is based on MongoDB shell (mongosh) syntax. The dataset contains 50 documents with item ID, dish name, category, price and rating, with 10 dishes in each of the five categories.

INSERTING MENU RECORDS

```
db.menu.insertMany([
  {itemid:301, dish:"Paneer Tikka", category:"Starters", price:280, rating:4.5},
  {itemid:302, dish:"Veg Spring Rolls", category:"Starters", price:180, rating:4.0},
  {itemid:303, dish:"Chicken Wings", category:"Starters", price:320, rating:4.3},
  ...
  {itemid:350, dish:"Iced Tea", category:"Beverages", price:100, rating:4.0}
])
```

All 50 menu documents are inserted successfully into the menu collection.

DISPLAYING RECORDS

```
db.menu.find()
```

The find() operation retrieves documents from the menu collection. The shell displays records in batches of 20 documents.

COMPARISON OPERATORS

Operator	Example Query	Purpose / Matches
$eq	db.menu.find({ rating: { $eq: 4.5 } })	Rated exactly 4.5 (5 results)
$gt	db.menu.find({ price: { $gt: 400 } })	Priced above 400 (1 result)
$lt	db.menu.find({ price: { $lt: 50 } })	Priced below 50 (3 results)
$gte	db.menu.find({ rating: { $gte: 4.6 } })	Rated 4.6 or more (10 results)
$lte	db.menu.find({ price: { $lte: 60 } })	Priced 60 or less (10 results)

MONGODB OPERATIONS DEMONSTRATED

UPDATE ONE

```
db.menu.updateOne({ itemid: 311 }, { $inc: { price: 10 } })
```

Raises the price of "Butter Chicken" by 10. The output shows matchedCount: 1 and modifiedCount: 1.

UPDATE MANY

```
db.menu.updateMany({ category: "Beverages" }, { $inc: { price: 10 } })
```

Raises the price of every Beverages item by 10. The output shows 10 documents matched and modified.

AND

```
db.menu.find({ $and: [ { rating: { $gte: 4.5 } }, { category: "Desserts" } ] })
```

Returns desserts rated 4.5 or higher (4 results).

OR

```
db.menu.find({ $or: [ { price: { $lte: 50 } }, { category: "Beverages" } ] })
```

Returns items priced 50 or less, or in the Beverages category (13 results).

SORT

```
db.menu.find().sort({ price: -1 }).limit(5)
```

Sorts by price in descending order and shows the five costliest dishes: Mutton Rogan Josh (480), Fish Curry (400), Butter Chicken (390), Chicken Biryani (350), Fish Fingers (340). Ascending order is obtained with 1.

LIMIT

```
db.menu.find().limit(3)
```

Displays only the first three documents.

SEARCH BY CATEGORY

```
db.menu.find({ category: "Main Course" })
```

Finds all dishes in the Main Course category (10 results).

DELETE ONE

```
db.menu.deleteOne({ itemid: 316 })
```

Deletes the single document with item ID 316 (Mutton Rogan Josh).

DELETE MANY

```
db.menu.deleteMany({ rating: { $lt: 4.0 } })
```

Deletes all dishes rated below 4.0; 5 documents are deleted.

COUNT

```
db.menu.countDocuments({})
```

Counts documents currently in the collection; the final result is 44.

RESULTS

The project successfully demonstrated the core NoSQL operations required for managing a restaurant menu in MongoDB. The original dataset contained 50 menu documents, and queries were executed to retrieve, filter, modify, delete and organise the records.

Operation	Result	Observation
Initial records inserted	50	50 menu documents inserted successfully.
updateOne()	1 matched / 1 modified	Price of item ID 311 increased by 10.
updateMany()	10 matched / 10 modified	Price increased by 10 for Beverages.
deleteOne()	1 deleted	Item ID 316 removed.
deleteMany()	5 deleted	Dishes rated below 4.0 removed.
Final count	44	Count reported after the deletion operations.

LEARNING OUTCOMES

- Understanding of the MongoDB document model and NoSQL approach.
- Practical use of database and collection creation commands.
- Hands-on experience with CRUD operations.
- Understanding of comparison and logical operators.
- Ability to sort, limit, search and count MongoDB documents.
- Understanding of how query results change after update and delete operations.

PROJECT RESULT

The project demonstrates a complete workflow for managing a restaurant menu in MongoDB, from database creation and insertion through querying, updating, deletion and final document counting.

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Restaurant Menu Management System containing dish information was created and different MongoDB operations were performed on the data.

Through this project I learned how to insert, retrieve, update and delete documents, how to sort data, how comparison operators work, and how logical operators such as $and and $or combine conditions. Overall, the project helped me understand how MongoDB can be used to manage and query menu data efficiently.

FUTURE SCOPE

- Develop a web-based ordering interface connected to MongoDB.
- Add authentication and role-based access for managers and staff.
- Add customer records, order tracking and bill generation modules.
- Add dashboards and charts for popular dishes and sales analytics.
- Deploy the database using MongoDB Atlas and the application on a cloud platform.
- Add validation, indexing and aggregation queries for larger datasets.

REFERENCES

- MongoDB Documentation – MongoDB Query and CRUD concepts.
- MongoDB documentation on comparison and logical query operators.
- MongoDB documentation on update, delete, sort, limit and count operations.

THANK YOU!

Restaurant Menu Management System Using MongoDB

ASHISH PATEL
BCA (DS & AI) | Roll No. 1250258144
Babu Banarasi Das University, Lucknow